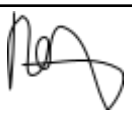
Declaration of Original Work for CT2003 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Assignment Group	Signature /Date
AARON LIM JING KAI	1	 14/11/25
ANG WEE HWA	1	 14/11/25
AQILA BINTE MOHAMED JAHBAR GANI	1	 14/11/25
BOWEN YIP WEI DE	1	 14/11/25
CHUA ZHEN YUN SERAPHINA	1	 14/11/25

Important notes:

1. Name must EXACTLY MATCH the one printed on your Matriculation Card.
2. Student Code of Academic Conduct includes the latest guidelines on usage of Generative AI and any other guidelines as released by NTU.

Table of Contents

Declaration of Original Work for CT2003 Assignment....1
Table of Contents....2
1) Design Considerations....3
1.1 Approach Taken....3
1.2 Design Principles Used (SOLID)....4
1.2.1 Single Responsibility Principle (SRP)....4
1.2.2 Open/Closed Principle (OCP)....6
1.2.3 Liskov Substitution Principle (LSP)....7
1.2.4 Interface Segregation Principle (ISP)....8
1.2.5 Dependency Inversion Principle (DIP)....9
1.3 Assumptions Made....10
1.4 Utilization of Object-Oriented Concepts....11
1.4.1 Encapsulation and Abstraction (Classes and Objects)....11
1.4.2 Inheritance and Method Overriding (Classes and Objects)....11
1.4.3 Polymorphism....12
1.4.4 Exception Handling....13
1.5 Reusability, Extensibility, and Maintainability....14
2) UML Class Diagram....15
2.1 Relationships....16
2.1.1 Inheritance....16
2.1.2 Interface Realisation....16
2.1.3 Composition (Owns)....16
2.1.4 Aggregation (Not Owned)....16
2.1.5 Association (Simple Structural Link)....17
2.1.6 Dependency (Uses)....17
2.2 Multiplicities....17
3) Testing....18
3.1 Testing Objectives....18
3.2 Testing Methodology....18
3.3 Summary of Test Case Scenarios....19
3.4 Testing Conclusion....19
4) Additional Feature - "Student's Appeal"....20
4.1 Assumptions Made....20
4.2 Sequence Diagrams....20
4.3 Design & Integration with Existing Code....21
4.4 Extensibility in Action....21
5) Reflection....22
5.1 Difficulties Encountered....22
5.2 Knowledge Gained....22
5.3 Future Improvements....23
6) Conclusion....26
References....27

1) Design Considerations

1.1 Approach Taken

Our group adopted a layered CLI architecture that separates functions into four clear bands:

- Domain model (entities): User (with Student, Staff, CompanyRep), InternshipOpportunity, and Application.
- Business services: PortalService (opportunities, applications, rules/workflows), AuthService (user auth, staff approvals), and PreferenceService (persistent list preferences via ListPreferences).
- Persistence adapters: OppAppCsv (opportunities & applications) and CsvUtil (users) handle CSV I/O, keeping storage details out of the domain/services
- Presentation (CLI): role-specific menus StudentMenu, CompanyRepMenu, StaffMenu, with SimpleInput and TablePrinter as I/O utilities.

Main Benefits of the Approach:

- Clean separation of concerns keeps menus thin and testable.
- A single in-memory repository (PortalService) keeps the flow simple for a CLI app, with CSV as the canonical store (assignment-friendly, zero external deps).
- Strategy-like design for filters/sorts makes it easy to add more columns without touching the UI code.

1.2 Design Principles Used (SOLID)

Our group focused on detecting violations of the Principles and thus improving our code designs by adhering to the SOLID Principles closely.

1.2.1 Single Responsibility Principle (SRP)

Examples of Violation in Original Version:

- A single Godclass UserService that handles anything from user authentication, processing applications and opportunities and even reading/writing CVS

1. UserService (Authentication Service)

```
// UserService.java  (SRP violation: auth inside UserService)
public boolean login(String idOrEmail, String password) {
    for (Student s : students) {
        if ((s.getStudentId().equalsIgnoreCase(idOrEmail) || s.getEmail().equalsIgnoreCase(id
            && Objects.equals(s.getPassword(), password)) { currentUser = s; return true; }
    }
    for (Staff st : staff) {
        if ((st.getStaffId().equalsIgnoreCase(idOrEmail) || st.getEmail().equalsIgnoreCase(id
            && Objects.equals(st.getPassword(), password)) { currentUser = st; return true; }
    }
    for (CompanyRep r : reps) {
        if (r.getEmail().equalsIgnoreCase(idOrEmail) && r.isApproved()
            && Objects.equals(r.getPassword(), password)) { currentUser = r; return true; }
    }
    return false;
}
```

2. UserService (Applications/Opportunities Processing)

```
// UserService.java  (SRP violation: domain rules inside same class)
public void createOpportunity(CompanyRep rep, String company, String title, int slots) {
    String id = "OPP-" + (opportunities.size() + 1);
    InternshipOpportunity o = InternshipOpportunity.draft(rep.getEmail(), company, title, "", "An
    opportunities.add(o);
    saveAll(); // directly triggers persistence
}

public boolean applyTo(Student s, String oppId) {
    InternshipOpportunity o = findOpp(oppId);
    if (o == null || !o.isApproved() || !o.isOpenNow() || o.isFilled()) return false;
    applications.add(Application.newPending(s.getStudentId(), oppId));
    saveAll(); // persistence coupled to business action
    return true;
}
```

3. UserService (CVS Persistence Service)

```
// UserService.java  (SRP violation: CSV I/O in same class)
private void reloadAll() {
    ensureFileWithHeader("students.csv", "StudentID,Name,Major,Year,Email,Password");
    try (BufferedReader br = Files.newBufferedReader(Path.of("students.csv"), StandardCharsets.UTF
        br.readLine(); // header
        for (String line; (line = br.readLine()) != null; )
            if (!line.isBlank()) students.add(parseStudent(line));
    } catch (IOException e) { System.err.println("Load students failed: " + e.getMessage()); }

    // ...similar blocks for staff, reps, opportunities, applications...
}
```

- A class should have only one reason to change, however our UserService has at least three different types of functions mixed in.
- It has more than 3 reasons to change (Auth policy, Domain policy, Persistence schema), resulting in low cohesion, hard testing (needs files to test auth), and ripple effects.

Proposed Solution:

- Segregating UserService into focused single-responsibility components
    1. UI / CLI:
        - StudentMenu, StaffMenu, CompanyRepMenu, SimpleInput, TablePrinter
        - Only read input and print output; delegate all rules to services instead
    2. Domain services:
        - AuthService that focuses on authentication & user directory (find/login/change password/approve)
        - PortalService that focuses on internship/application rules (create/edit/delete, apply/accept, filter/sort, appeals)
    3. Persistence helpers:
        - CsvUtil handles Students/Staff/CompanyReps CSV I/O
        - OppAppCsv handles Opportunities/Applications CSV I/O
        - PreferenceService handles sorting/filtering preferences CSV I/O

Advantages of the Change:

- Change isolation (one reason to change)
    - Auth policies touch only AuthService, opportunity rules touch only PortalService, and CSV schema/storage changes touch only CsvUtil/OppAppCsv. Ripple effects are minimized.
- Easier to swap technologies
    - Replacing CSV with a DB (JDBC/ORM) is localized to CsvUtil/OppAppCsv, and the domain services and menus remain unchanged.
- Extensibility for new features
    - Adding new features in the future causes less refactoring of codes due to specialization of roles, across various classes

1.2.2 Open/Closed Principle (OCP)

Examples of Violation in Original Version:

- Menus hard-wired to call the concrete PreferenceService

```
class StudentMenu {
    private final PortalService portal = PortalService.getInstance();
    // Direct dependency on concrete class (no interface)
    private final PreferenceService prefs = PreferenceService.getInstance();

    void listOpen(Student me) {
        // Menu calls the concrete CSV-backed service directly
        ListPreferences p = prefs.get(me, "student_internships");  // concrete call

        List<InternshipOpportunity> base = portal.listOpenForStudent(me, true);
        List<InternshipOpportunity> view = portal.applyOppPrefs(base, p);
        TablePrinter.printMenu("Internships", header, rows(view));
    }
}
```

- Callers depend on a concrete CSV implementation, thus if we were to change the Preference Persistence Service (CSV → file per user → DB → encrypted/cached), it forces edits in every menu that calls PreferenceService, making modules susceptible to modification.
- We will not be able to change what the modules do, without changing the source code of the modules, blatantly violating OCP.

Proposed Solution:

- Introducing a new interface PreferencesStore:

```
public interface PreferencesStore {
    ListPreferences get(User u, String view);
    void put(User u, String view, ListPreferences p);
    String key(User u, String view);
}
```

- Making PreferenceService implementing it instead:

```
public class PreferenceService implements PreferencesStore {
    // existing implementation (CSV-backed) …
}
```

- Making Menus to call the implementations instead

```
public class StudentMenu {
    private final AuthService auth;
    private final SimpleInput in;
    private final PortalService portal = PortalService.getInstance();

    // Note the type is the interface (PreferencesStore); impl can change freely.
    private final PreferencesStore prefs = PreferenceService.getInstance();
```

Advantages of the Change:

- Callers (menus) now depend on PreferencesStore (the abstraction) while PreferenceService hides the storage details.
- If we want to switch from CSV cache to DB/encrypted/Redis, you only update PreferenceService (or swap implementation in wiring).
- Menus remain closed to modification, achieving OCP.

1.2.3 Liskov Substitution Principle (LSP)

Examples of Violation in Original Version:

- Strengthened Precondition + Weakened Postcondition (In Sub-Class)

```
abstract class User {
    private String password;
    public boolean passwordMatches(String attempt) {
        return Objects.equals(password, attempt);    // contract: returns boolean, no exceptions
    }
}
```

```
class CompanyRep extends User {
   private boolean approved;
   @Override
   public boolean passwordMatches(String attempt) {
       if (!approved) {                          // ✗ strengthens precondition
           throw new IllegalStateException("Not approved"); // ✗ changes contract (throws)
       }
       return super.passwordMatches(attempt);
   }
}
```

- Functions that work for User, must also work for its subtypes
- By throwing for unapproved representatives, CompanyRep strengthens preconditions and weakens the Post Condition, thus a CompanyRep can't be safely substituted everywhere a User is expected, violating LSP.

Proposed Solution:

- Keep User.passwordMatches(...) uniform and side-effect-free across all subclasses.

```
// AFTER (current pattern)
abstract class User {
    private String password;
    public boolean passwordMatches(String attempt) {
        return Objects.equals(password, attempt);    // uniform, no throws
    }
}

class CompanyRep extends User {
    private boolean approved;
    public boolean isApproved() { return approved; } // extra state, no contract change
}
```

- Put role-specific checks (like "rep must be approved") outside the User hierarchy, for example we put it in AuthService.

```
public boolean attemptLogin(String idOrEmail, String password) {
    User u = findByIdOrEmail(idOrEmail);           // Student / Staff / CompanyRep
    if (u == null) return false;

    if (!u.passwordMatches(password)) return false; // same contract for all Users

    // role-specific constraint handled by the service, not by overriding the base method
    if (u instanceof CompanyRep rep && !rep.isApproved()) {
        // return false or mark NOT_APPROVED via your result type
        return false;
    }
    return true;
}
```

Advantages of the Change:

- True substitutability: Code written against User works for Student, Staff, or CompanyRep identically, no hidden throws or extra preconditions.
- Stable contracts. passwordMatches always returns a boolean and never throws; callers don't need try/catch for specific subtypes.

1.2.4 Interface Segregation Principle (ISP)

Examples of Violation in Original Version:

- A single utility class CsvUtil handled ALL CSV operations.
- However, Menus (StudentMenu, StaffMenu, CompanyRepMenu) only needed preferences but had to import the FAT CsvUtil.
- AuthService only needed user directories, yet was coupled to opportunities, applications, and preferences.
- PortalService only needed opportunities/applications, yet was coupled to users and preferences.

```
// Menus (only need prefs)
ListPreferences p = CsvUtil.getUserPrefs(me, VIEW_ID);
CsvUtil.putUserPrefs(me, VIEW_ID, p);

// AuthService (only needs user directories)
List<Student> sts = CsvUtil.loadStudents("students.csv");

// PortalService (only needs opps/apps)
var opps = CsvUtil.readOpportunities();
var apps = CsvUtil.readApplications();
```

- Multiple classes were forced to depend on methods they did not use inside CsvUtil

Proposed Solution:

- Segregate capabilities into focused modules that clients only use what it needs.

4. Preferences (PreferenceStore/PreferenceService)

```
public interface PreferencesStore {
    ListPreferences get(User u, String view);
    void put(User u, String view, ListPreferences p);
    void reset(User u, String view);
}

/** CSV-backed implementation (user_prefs.csv only). */
public class PreferenceService implements PreferencesStore { /* ... */ }
```

```
private final PreferencesStore prefs = PreferenceService.getInstance();
ListPreferences p = prefs.get(me, VIEW_ID);
prefs.put(me, VIEW_ID, p);
```

5. Opportunities/Applications (OppAppCsv)

```
// PortalService remains on opp/app CSV only
List<InternshipOpportunity> opps = OppAppCsv.readOpportunities();
List<Application> apps = OppAppCsv.readApplications();
```

6. User directories (Keeping in CsvUtil)

Advantages of the Change:

- Change-ripple reduced:
    - Changing user CSV columns no longer affects menus or PortalService.
    - Changing opp/app storage does not affect menus or AuthService.
    - Preferences evolution does not touch user or opp/app code.

1.2.5 Dependency Inversion Principle (DIP)

Examples of Violation in Original Version:

- High-level modules (Menus) depended on a concrete, low-level class (PreferenceService) and called its singleton:

```
ListPreferences p = PreferenceService.getInstance().get(me, "student_internships");
PreferenceService.getInstance().put(me, "student_internships", p);
```

- StudentMenu, StaffMenu, and CompanyRepMenu are high-level modules while PreferenceService is a low-level module (file-backed CSV persistence for user list preferences).
- This must be changed, because any change to how preferences are stored (CSV → JSON → DB), would force code changes and re-compilation in all menus, a tight coupling, resulting in harder substitution and testing.

Proposed Solution:

- Introducing a new interface PreferencesStore:

```
public interface PreferencesStore {
    ListPreferences get(User u, String view);
    void put(User u, String view, ListPreferences p);
    void reset(User u, String view);
}
```

- Making PreferenceService implementing it instead:

```
public class PreferenceService implements PreferencesStore {
    // existing implementation (CSV-backed) …
}
```

- Refactor high-level modules (menus) to the abstraction:

```
private final PreferencesStore prefs = PreferenceService.getInstance();

ListPreferences p = prefs.get(me, VIEW_ID);
prefs.put(me, VIEW_ID, p);
```

Advantages of the Change:

- Menus (StudentMenu, StaffMenu, CompanyRepMenu) now depend on PreferencesStore (an abstraction), not PreferenceService (a detail).
- PreferenceService (detail) now implements PreferencesStore (abstraction). The direction of dependency is inverted.
- Substitutability: swap CSV storage for JSON/DB or an in-memory stub without touching menu code.
- Lower ripple: changes to persistence format/classes no longer cascade into UI flows.

1.3 Assumptions Made

1. Eligibility Rules:
    - Year 1–2 -> Basic level only
    - Year3-> Any level
    - Major preference is matched by string comparison
2. Application Limits:
    - Each student may have at most 3 active applications
    - Accepting one offer withdraws/invalidates the rest
3. Opportunity Window:
    - An opportunity is “open” if isOpenNow() (based on openingDate/closingDate) and not filled
4. Uniqueness:
    - InternshipOpportunity.id is unique
    - Each Application is unique by (studentId, opportunityId)
5. Student Internship Appeals (Additional Feature):
    - One appeal per unsuccessful application
    - Students with accepted applications will be ineligible to Appeal
    - Company Representative reviews Appeals with Appeal Message
    - If approved, status flips to “Successful” and Students can proceed to accept
    - If rejected, status flips to “Unsuccessful” and appealed applications can no longer appeal in the future
6. Persistence:
    - CSV files are the source of truth
    - Upon mutating actions, in-memory lists are saved back immediately
7. Session model:
    - Single-user CLI session; concurrency and locking are out of scope (assignment constraint)

1.4 Utilization of Object-Oriented Concepts

1.4.1 Encapsulation and Abstraction (Classes and Objects)

Domain entities keep state private and guard their invariants:

- Example: InternshipOpportunity — window & capacity rules live with the object
- Relevant Code:

```
private String openingDate;
private String closingDate;
private int slots;
private int confirmed;

public boolean isOpenNow() {
    var open  = LocalDate.parse(openingDate);
    var close = LocalDate.parse(closingDate);
    var today = LocalDate.now();
    return !isFilled() && !today.isBefore(open) && !today.isAfter(close);
}

public boolean isFilled() {
    return confirmed >= slots;
}

public void increaseConfirmed() {
    if (isFilled()) throw new IllegalStateException("All slots filled");
    confirmed++;
}
```

- Advantages of Encapsulation:
    - Dates and counters are not exposed directly.
    - Validity (open window, capacity ceiling) is checked inside the class whenever state changes.
    - Other parts of the program call isOpenNow() / increaseConfirmed() instead of duplicating rule logic, reusable codes

1.4.2 Inheritance and Method Overriding (Classes and Objects)

Role hierarchy with one base, three specialisations:

- Relevant Code:

```
// User.java (base)
public class User {
    private String userId, name, email, password, role;
    // getters/setters, passwordMatches(...), toString() …
}

// Student.java
public class Student extends User {
    private String major; private int year;
    @Override public String toString() {
        return getStudentId() + " - " + major + " (Y" + year + ")";
    }
}

// Staff.java
public class Staff extends User {
    private String department;
    @Override public String toString() { /* staff-specific summary */ }
}

// CompanyRep.java
public class CompanyRep extends User {
    private String company, department, position;
    private boolean approved;
    @Override public String toString() { /* rep-specific summary */ }
}
```

- Advantages of Inheritance and Method Overriding:
    - Common identity/credentials live in User; subclasses only add what's unique.
    - Anywhere we handle a User (e.g., after login), we can branch to role-specific menus without casting scattered throughout the code
    - Role behaviour (e.g., toString() output) is overridden where it belongs, which refers as Method overriding
    - Allows future extension of more User Groups like Alumni, Parents

1.4.3 Polymorphism

Polymorphism lets one call site dispatch different concrete behaviours at runtime depending on the object or value supplied, and these are the types we used:

1. Subtype Polymorphism via Java types (e.g., Comparator<InternshipOpportunity>)
2. Strategy Polymorphism by selecting a Sorting or Filtering strategy from a Column choice, so callers don't hard-code behaviour

1. Strategy selection for Sorting (Runtime Behaviour Swap)

- A single method returns different comparators based on the chosen column. Callers don't know how items are compared; they only request "sort by this column".
- Relevant Code:

```
// PortalService.java
public Comparator<InternshipOpportunity> comparatorFor(Column c) {
    switch (c) {
        case TITLE:
            return Comparator.comparing(InternshipOpportunity::getTitle, String.CASE_INSENSITIVE_C
        case COMPANY:
            return Comparator.comparing(InternshipOpportunity::getCompanyName, String.CASE_INSENSI
        case OPENING_DATE:
            return Comparator.comparing(o -> LocalDate.parse(o.getOpeningDate()));
        case EMPTY_SLOTS:
            return Comparator.comparingInt(o -> o.getSlots() - o.getConfirmed()).reversed();
        default:
            return Comparator.comparing(InternshipOpportunity::getId);
    }
}

public List<InternshipOpportunity> applySort(List<InternshipOpportunity> base, Column c) {
    return base.stream().sorted(comparatorFor(c)).toList();
}
```

- ApplySort(…, c) calls the same method but gets a different comparator object (strategy) depending on c

2. Strategy selection for filtering (single call site, many behaviours)

- Filtering uses a unified key extractor. The UI passes a Column and a keyword; PortalService picks the right way to get text for matching
- Relevant Code:

```
// PortalService.java
public String valueFor(InternshipOpportunity o, Column c) {
    switch (c) {
        case TITLE:        return o.getTitle();
        case COMPANY:      return o.getCompanyName();
        case LEVEL:        return o.getLevel();
        case PREF_MAJOR:   return o.getPreferredMajor();
        case STATUS:       return o.isApproved() ? "APPROVED" : (o.isFilled() ? "FILLED" : "PENDI
        case TOTAL_SLOTS:  return Integer.toString(o.getSlots());
        case EMPTY_SLOTS:  return Integer.toString(o.getSlots() - o.getConfirmed());
        default:           return o.getId();
    }
}

public List<InternshipOpportunity> applyFilter(List<InternshipOpportunity> base, Column c, String
    String kw = keyword.toLowerCase();
    return base.stream()
        .filter(o -> valueFor(o, c).toLowerCase().contains(kw))
        .toList();
}
```

- valueFor plays the strategy for "Extracting a searchable value within a Column," which changes per column, but the filter pipeline remains identical.
- Easier Testability for new strategies, better Consistency across screens/menus

1.4.4 Exception Handling

CsvUtil and PortalService is an excellent example for exhibiting Exception Handling, particularly to prevent crashes due to reading or writing to CVS files:

1. Proactive file creation (happy-path guard)
    - Every read starts with ensureFileWithHeader(filename, header). If the file is missing, it's created with the correct header using a try-with-resources writer
    - Relevant Code:

```
private static void ensureFileWithHeader(String filename, String header) throws IOException {
    Path p = Path.of(filename);
    if (!Files.exists(p)) {
        try (BufferedWriter bw = Files.newBufferedWriter(p, StandardCharsets.UTF_8)) {
            bw.write(header);
            bw.newLine();
        }
    }
}
```

2. All CsvUtil public methods declared throws IOException
    - AuthService.reload() wraps each call in try/catch: on failure it prints an error to System.err and replaces the in-memory list with an empty list (so the app keeps running).
    - PortalService does the same for OppAppCsv (opportunities/applications).
    - Relevant Code:

```
// CsvUtil.java
public static void writeStudents(List<Student> students) throws IOException {
    try (BufferedWriter bw = Files.newBufferedWriter(Path.of(STUDENT_CSV), StandardCharsets.UTF_8)
        // normalized header
        bw.write(joinCsv("StudentID", "Name", "Major", "Year", "Email", "Password"));
        bw.newLine();
        for (Student s : students) {
            bw.write(joinCsv(
                    nz(s.getStudentId()),
                    nz(s.getName()),
                    nz(s.getMajor()),
                    Integer.toString(s.getYear()),
                    nz(s.getEmail()),
                    nz(s.getPassword())
            ));
            bw.newLine();
```

3. PortalService.reload() catches IOException from its CSV helper
    - Relevant Code:

```
// PortalService.java
public final void reload() {
    opportunities.clear();
    applications.clear();
    try { opportunities.addAll(OppAppCsv.readOpportunities()); }
    catch (IOException e) { System.err.println("Failed to read opportunities.csv: " + e.getMessage
    try { applications.addAll(OppAppCsv.readApplications()); }
    catch (IOException e) { System.err.println("Failed to read applications.csv: " + e.getMessage(
}
```

    - Even AuthService.reload() follows the same pattern (three try/catch blocks for students, staff, reps—on failure it logs and replaces with an empty list).
    - This layering "CsvUtil throws, services catch" ensures missing/locked/malformed CSVs never crash the CLI and degrades gracefully (logs + keeps running)

1.5 Reusability, Extensibility, and Maintainability

1.5.1 Reusability

- Layered design enables reuse across UIs. Menus are thin; business rules live in PortalService/AuthService; CSV adapters (OppAppCsv, CsvUtil) encapsulate storage. The same services can power a future GUI or REST API without code duplication.
- Shared utilities. SimpleInput and TablePrinter are UI-agnostic helpers reused by StudentMenu, StaffMenu, and CompanyRepMenu.
- Central filter/sort pipeline. A single set of methods (valueFor, comparatorFor, applyFilter, applySort) in PortalService is reused by all roles; no menu re-implements filtering or sorting.
- Domain predicates. Methods like Application.isPending() / isSuccessful() / isAppealing() are reused everywhere instead of scattering string comparisons.

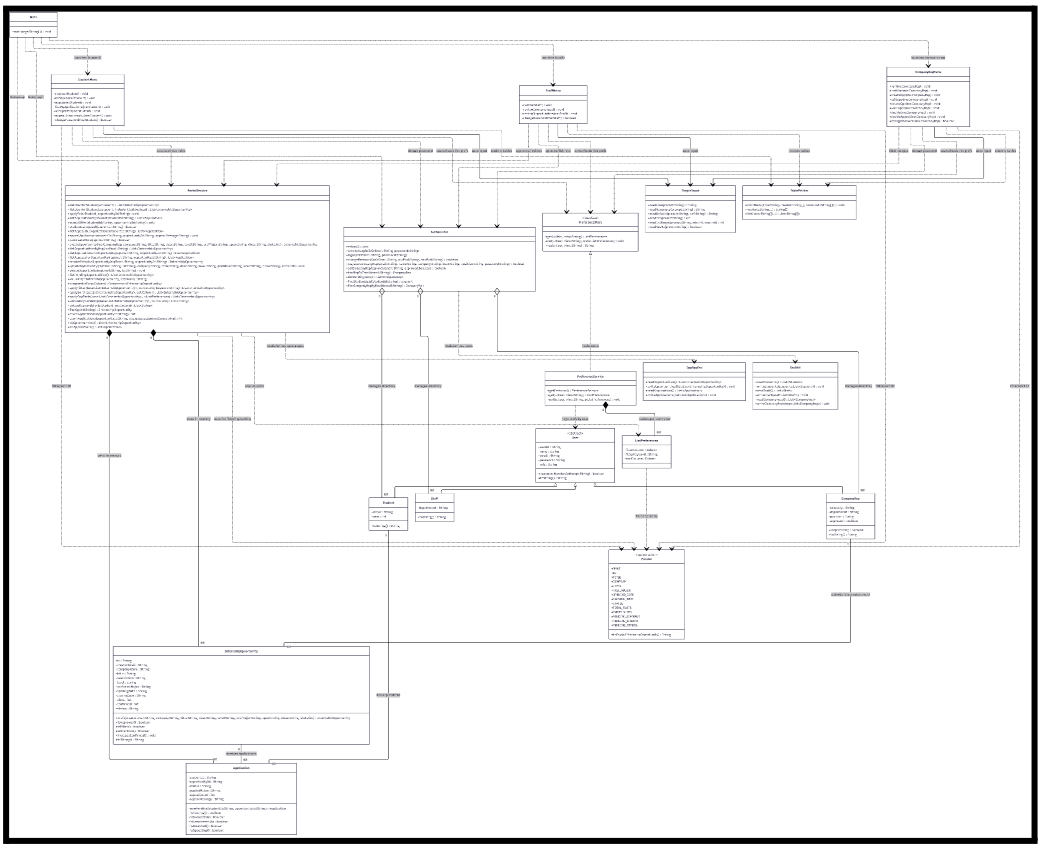
1.5.2 Extensibility

- New preference backends with zero caller changes. Menus depend on the PreferenceStore interface, realized by PreferenceService. Swapping CSV to JSON/DB/encrypted/cache only changes the implementation/wiring, not the callers (OCP + DIP).
- Add a new listing column. Extend Column and map it once in PortalService (valueFor + comparatorFor). All menus pick it up automatically through applyOppPrefs.
- Introduce new user roles. Subclass User (e.g., Alumni) and add role-specific behavior outside base contracts (e.g., policies in AuthService). Existing flows continue to work (LSP).
- Evolve business rules. Capacity, eligibility, and appeal policies reside in PortalService; changes don't spill into menus or persistence.
- One good example would be our Student Appeal Feature: It plugged in without changing unrelated modules because validation/rules are centralized and persistence is behind adapters.

1.5.3 Maintainability

- Single responsibility per class. UI, domain rules, and persistence are separated (SRP), reducing the reasons any file must change.
- Closed callers, open extensions. Menus call abstractions (PreferenceStore) and service entry points; adding features is done by adding code in the right module, not editing many places (OCP).
- Stable base contracts. User.passwordMatches(...) has a uniform contract; role-specific checks (e.g., rep approval) live in AuthService, preserving substitutability (LSP).
- Smaller change surface. One authority for rules (PortalService) and one for auth (AuthService) means less duplication and fewer inconsistencies.
- Graceful failure, simpler debugging. CSV helpers throw; services catch and degrade to safe defaults (empty lists + error messages). The CLI remains responsive; issues are localized.
- Testability. You can unit-test PortalService with in-memory data and swap a fake PreferenceStore to test menus without touching the file system (ISP/DIP help here).

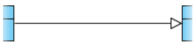
2) UML Class Diagram



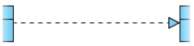
Refer to the Full PNG File for the Detailed Diagram

2.1 Relationships

2.1.1 Inheritance

- Symbol: 
- User is an abstract superclass
- Student, Staff, and CompanyRep specialize it (User <|-- Student/Staff/CompanyRep).
- Allows encapsulation of User and even extension of specialized capabilities for more users in future.

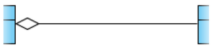
2.1.2 Interface Realisation

- Symbol: 
- PreferenceStore an interface, realized by PreferenceService (PreferenceStore <|.. PreferenceService).
- Menus/services depend on the abstraction.
- Thus Implementation can be swapped without touching clients.

2.1.3 Composition (Owns)

- Symbol: 
- PortalService owns InternshipOpportunity and Application in memory
- These parts are created, contained, and discarded under the service's control
- If service clears or is replaced, the contained objects become unreachable and dies too

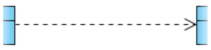
2.1.4 Aggregation (Not Owned)

- Symbol: 
- AuthService with Student, Staff, and CompanyRep, where it manages the directory directly
- AuthService maintains navigable collections but does not own user lifecycles (users are persisted via CSV and conceptually outlive the service instance).

2.1.5 Association (Simple Structural Link)

- Symbol: 
- Student have applications with Application.
- InternshipOpportunity receives Application
- CompanyRep submits via “creatorEmail” to InternshipOpportunity
- These are domain links with business meaning but no ownership transfer.

2.1.6 Dependency (Uses)

- Symbol: 
- PortalService and AuthService uses OppAppCsv to Read/Write to User CVS files
- StaffMenu, StudentMenu and CompanyRepMenu uses PreferenceStore to Read/Write to user preference CVS file
- This is most commonly used relationship we have in our program design

2.2 Multiplicities

2.2.1 Domain associations

- Student "1" to "0..*" Application
- One student can have zero or many applications while each application links to exactly one student.

2.2.2 Compositions

- PortalService "1" to "0..*" InternshipOpportunity
- PortalService "1" to "0..*" Application
- PreferenceService "1" to "0..*" ListPreferences
- The service instance is the single whole and it contains zero or many parts at runtime.

2.2.3 Aggregations

- AuthService "1" to "0..*" Student/Staff/CompanyRep, managing its directory
- A single AuthService (singleton) maintains zero or many users of each type, but users are not life-cycle bound to the service

3) Testing

Overview

Comprehensive testing was conducted across three user perspectives: Student, Company Representative, and Career Center Staff. This ensured that the Internship Management System operates as intended and provides a smooth, reliable workflow from internship creation to student placement.

3.1 Testing Objectives

The primary testing objectives were to:

- Verify that each user can perform their respective actions successfully.
- Ensure that system responses, notifications, and error messages are triggered correctly.
- Confirm that data consistency and access control are maintained across all user perspectives.
- Test boundary and edge cases, particularly for appeal handling, application limits, and slot management.

3.2 Testing Methodology

The system was developed using Java and tested through the Eclipse IDE environment.

Given that this project involves a console-based system, manual testing was selected as the most suitable approach. This method allowed the team to directly observe program behavior and identify potential user-experience issues or logical errors that may not be captured through automation.

All test cases were executed manually by running the program and verifying both the console outputs and data updates. Test data were pre-initialized using CSV files containing sample records for students, company representatives, and internship opportunities. Each test case was designed and executed to validate the logical flow and expected outcomes based on the storyboard and system requirements.

The testing process included functional testing (to verify feature correctness), boundary testing (to check limit conditions such as appeal message length and slot capacity), and exception testing (to ensure robust handling of invalid inputs). This ensured that the system remained reliable and maintained data integrity across all user interactions.

3.3 Summary of Test Case Scenarios

Testing focused on the following key categories:

Category	Key Features Tested	Overall Result
Authentication and User Management	Login (valid/invalid), Password change, Logout, Password persistence	**Passed**
Registration and Initialization	Company Rep registration, System CSV initialization, Invalid file handling	**Passed**
Student Features	List and apply for opportunities, Withdraw applications, View application status, Set preference list, Internship acceptance	**Passed**
Company Representative Features	Create/Edit/Delete/Close opportunities, Confirm slots, Manage student applications, Internship slot capacity	**Passed**
Career Centre Staff Features	Approve company rep registration, Review opportunities and applications	**Passed**
Appeal Features	Student eligibility checks, Submit appeal (bounds testing), Approve/Reject appeals, Slot handling after appeal	**Passed**

3.4 Testing Conclusion

All core test scenarios passed successfully. The system behaved as expected for normal and edge cases, including login, application submission, appeal handling, and slot management. No critical failures or data inconsistencies were observed. The testing confirms that the Internship Management System is robust, user-friendly, and meets the intended requirements for all three user perspectives.

All detailed test cases, including their inputs, expected outputs, and actual results, are documented in the accompanying Excel file titled “Test Cases.xlsx”.

**Please refer to Test Cases.xlsx for complete test cases conducted by our Group*

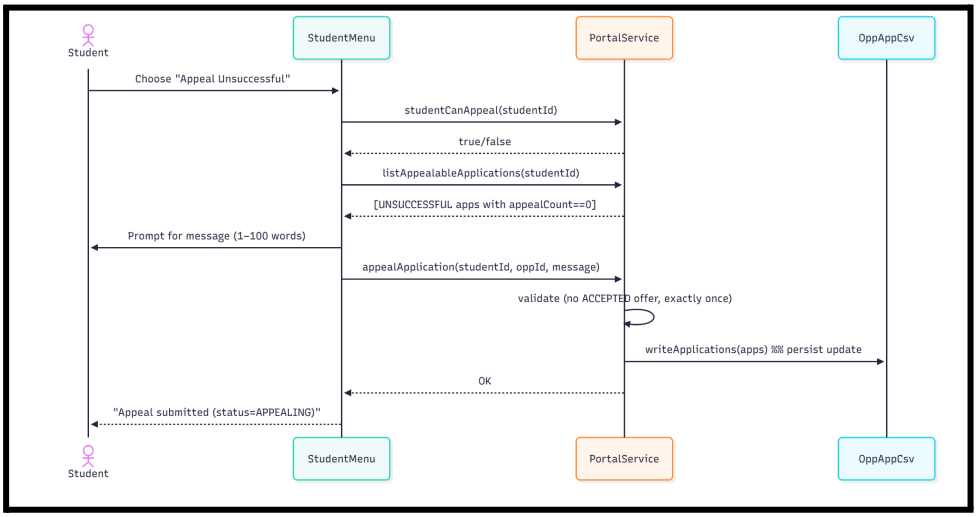
4) Additional Feature - “Student’s Appeal”

Our group has added a new additional function that allows Students to Appeal for “REJECTED” applications. As it is perfect for us to show the extensibility of our project yet enriching the functionalities of our internship program at the same time.

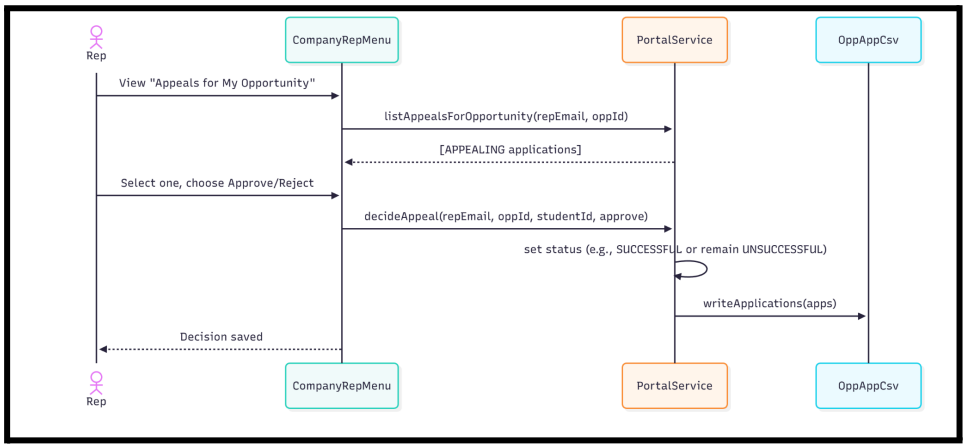
4.1 Assumptions Made

- Students can only use the Appeal Feature if they fulfill the following:
    1. Have at least 1 application with the status “REJECTED”
    2. Applications aforementioned has not been Appealed before
    3. Student does not have any applications with the status “ACCEPTED”
- Appeal message word count must be in between 1 word to 100 words

4.2 Sequence Diagrams



Student Appeal Flow



CompanyRep Review Flow

4.3 Design & Integration with Existing Code

- Domain model: Application holds appealCount and appealMessage, plus status predicates (isUnsuccessful(), isAppealing(), etc.).
- Business rules: PortalService exposes small, focused methods:
    - studentCanAppeal(studentId)
    - listAppealableApplications(studentId)
    - appealApplication(studentId, opportunityId, appealMessage)
    - listAppealsForOpportunity(repEmail, opportunityId)
- Menus (UI only):
    - StudentMenu.appealUnsuccessful(...) collects input and calls PortalService.
    - CompanyRepMenu.decideAppeal(...) lists pending appeals and calls the decision method.
- Data Persistence:
    - OppAppCsv persists the extra columns (AppealCount, AppealMessage) together with applications; no changes to user CSVs.
- Through our Layer Architecture, we only need to make the above minimal changes.

4.4 Extensibility in Action

- SRP: Menus stayed UI-only; rules live in PortalService; CSV is isolated in OppAppCsv. We added a handful of methods without touching unrelated modules.
- OCP: Menus didn't need new filter/sort logic; the appeal feature plugged into pre-existing service entry points.
- DIP/ISP: Preferences and other cross-cutting utilities remained untouched because callers already depend on interfaces/entry points.
- LSP: User hierarchy remained stable; no contract changes were needed to add appeals.
- In conclusion, due to our group adhering to SOLID Principles and constant usage of OOP concepts throughout our code, it allows us to extend and test new features with greater ease and flexibility.

5) Reflection

5.1 Difficulties Encountered

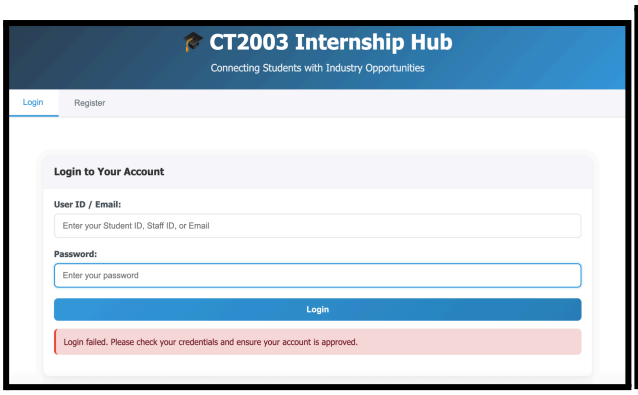
1. User-input edge cases:
    - Building resilient console flows (empty strings, out-of-range numbers, retries) demanded defensive helpers in SimpleInput and clear user feedback in menus
2. Understanding the SOLID Principles and building our program around it:
    - It requires us to deep dive into the concepts ourselves and actually understand how the SOLID principles work in real codes
    - We also have to constantly be self aware of not violating the principles when changing/adding codes
3. Data integrity across CSVs:
    - Keeping Opportunities–Applications–Users consistent (IDs, statuses, slot counts) required careful sequencing in PortalService and disciplined writes in OppAppCsv/CsvUtil.
4. Exception Handling during CSVs Read/Write:
    - We had to guard against missing/malformed CSVs and partial writes. The pattern “CSV throws; services catch and degrade” kept the CLI stable.

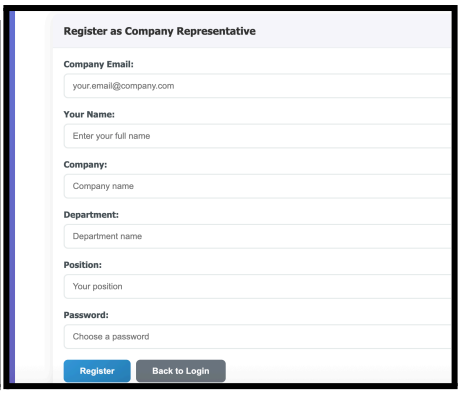
5.2 Knowledge Gained

1. Practical SOLID in a real codebase:
    - SRP by separating UI (*Menu) from rules (PortalService/AuthService) and persistence (OppAppCsv/CsvUtil);
    - OCP by making menus depend on PreferenceStore (not a concrete class);
    - LSP by keeping User.passwordMatches(...) contract uniform;
    - DIP/ISP by programming to the PreferenceStore interface.
2. Building extension points that scale:
    - Centralized filter/sort (valueFor, comparatorFor, applyOppPrefs) lets all roles share one pipeline.
    - Thus, adding new columns is localized and predictable.
3. Appropriate Usage of OOP Concepts:
    - When Inheritance, Method Overriding, Polymorphism are being used correctly, it reduces repetitive codes, allows extensibility and maintainability

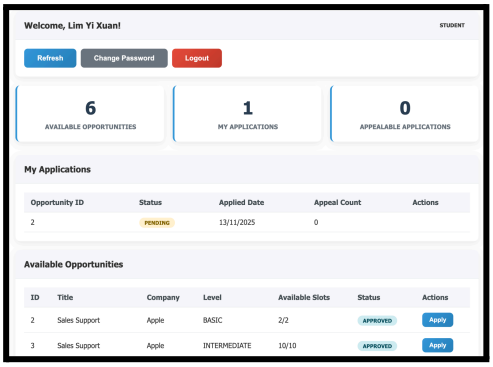
5.3 Future Improvements

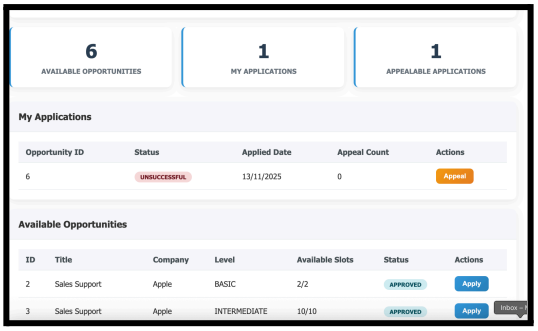
1. Moving beyond CSV
    - Migrate to SQLite/MySQL with DAOs or repositories. Only adapters change; services/menus stay intact. Add transactions for multi-record updates (e.g., accept-offer withdrawing others).
2. Parallelism & caching
    - Where applicable, parallelize expensive lookups and add in-memory caches with timed refresh for heavy lists—kept behind adapters so menus remain unchanged.
3. Security hardening
    - Replace plaintext passwords with salted hashing (e.g., PBKDF2/BCrypt) and centralize verification in AuthService. Mask sensitive outputs in logs.
4. Richer UX & automation
    - Add status notifications (email/SMS stubs), better keyword discovery, and bulk admin operations. Expose a REST layer to decouple UI from services.
5. Testing discipline
    - Introduce unit tests for PortalService rules and fake PreferenceStore for menu tests. Add CSV contract tests to prevent header/column drift.
6. User Interface - Frontend
    - Our Group has Utilised HTML, CSS and Javascript to create a user interface that works with our Backend.
    - Logging in with data from csv categorising them into student, staff and company Rep login with respective view, prompting wrong username / password / Register for Company Rep if no user is found.

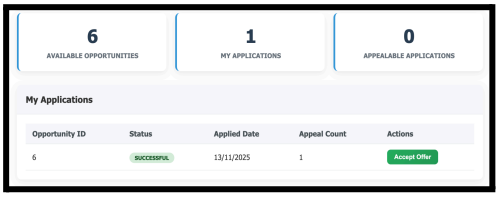


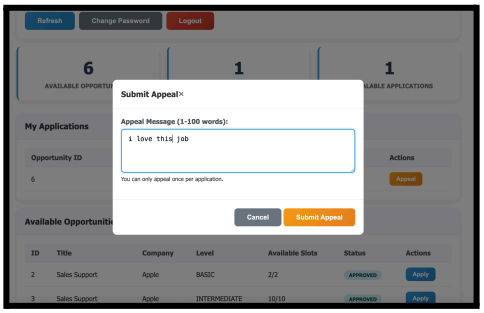


- Student Menu shows List of Internship Opportunities, Applications , Appeal Feature and Acceptance of Successful Internship

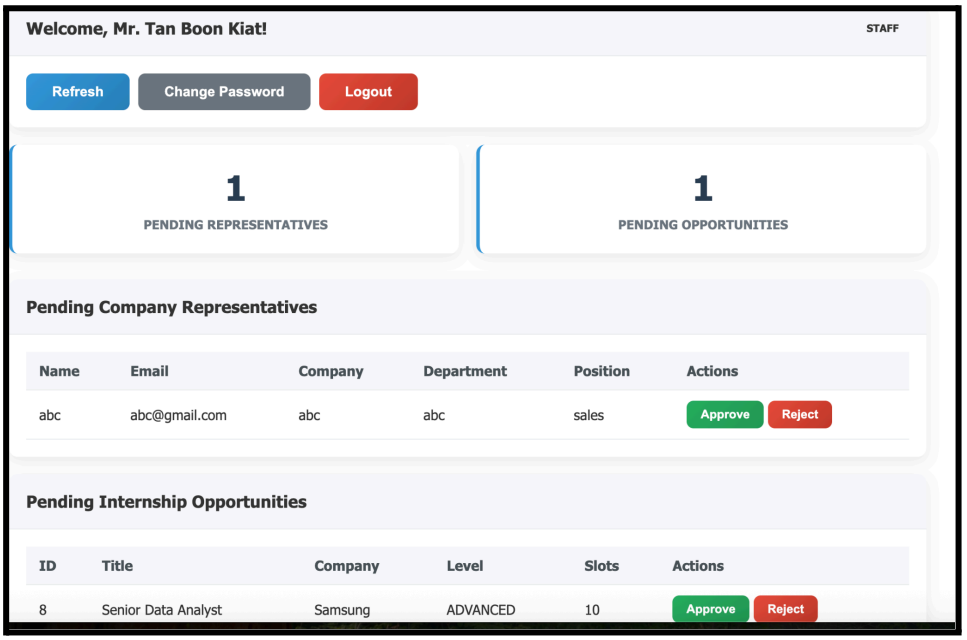




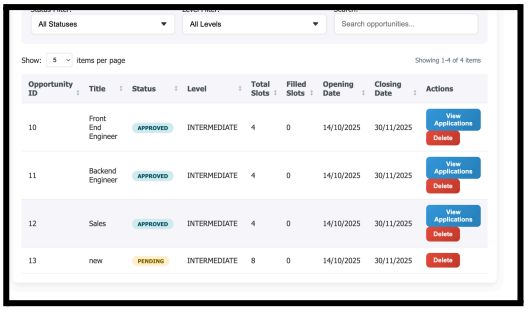




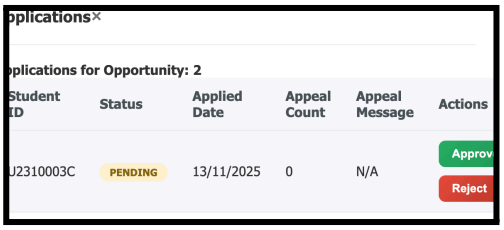
- Staff Menu that Lists Pending Company Reps and Internship Opportunities for Approvals and Rejections.



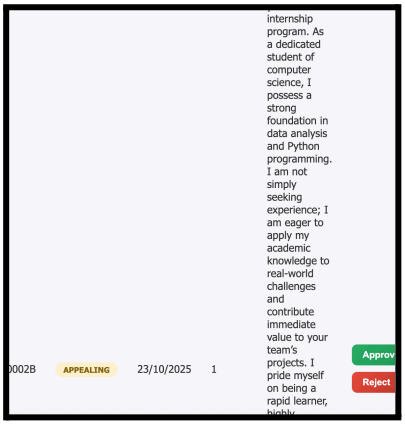
- CompanyRep Menu allows creation of new opportunities or deletion of existing opportunities

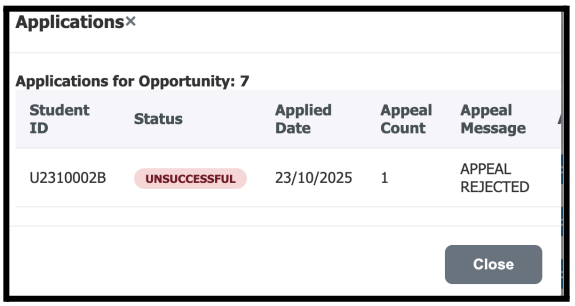


- CompanyRep Menu for approving/rejecting Applications

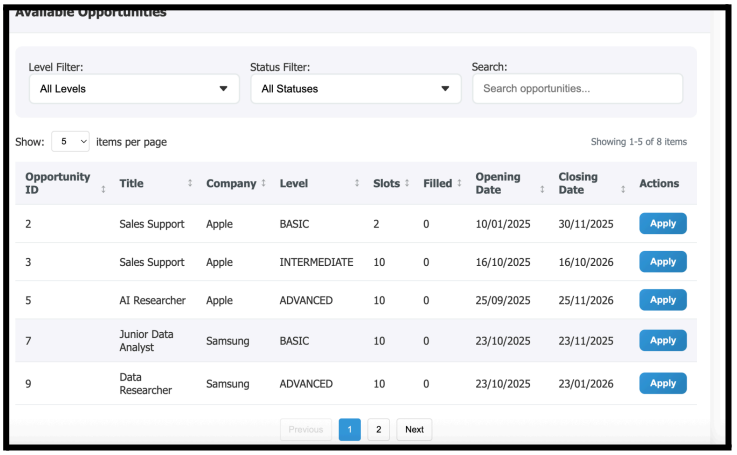


- CompanyRep Menu for reviewing Appeals





- Built-in Filtering Dropdown Menu and Sorting Buttons within GUI



6) Conclusion

This project set out to build a practical, CSV-backed internship portal for Students, Company Representatives, and Staff. Along the way we evolved a simple CLI into a cohesive system with clear responsibilities, persistence adapters, and user-centric features such as filtering/sorting with saved preferences and, importantly, the new Student Appeal workflow.

Architecturally, we applied SOLID principles to turn a tangle of concerns into clean layers: menus handle UI only; PortalService centralizes business rules; AuthService owns authentication and approvals; and OppAppCsv/CsvUtil isolate storage. We reduced ripple effects by programming to interfaces (e.g., PreferenceStore) and centralizing extension points (e.g., Column + PortalService's valueFor/comparatorFor). We preserved substitutability in the User hierarchy (LSP) by keeping base contracts stable and moving role policies into services. These choices improved testability, readability, and change isolation.

The strongest proof of design quality was how smoothly we delivered the Appeal feature late in the cycle. Because rules and persistence were already separated, we added focused methods (studentCanAppeal, listAppealableApplications, appealApplication, listAppealsForOpportunity) and minimal UI hooks, with no architecture rewrite. This demonstrated real extensibility rather than theoretical compliance.

Operationally, we hardened the CLI with defensive input handling and service-level error recovery for CSV I/O, ensuring the program degrades gracefully instead of crashing. The codebase is now positioned for pragmatic next steps—swapping CSV for a database, adding caching/parallelism where sensible, or exposing a REST interface—without touching the menus or rewriting core logic.

In short, we delivered a working system that meets the brief and a codebase that is easy to change. The combination of layered design, SOLID refactors, and well-placed abstractions gives us confidence that future features (analytics, notifications, richer review flows) can be added safely, quickly, and with minimal churn.

References

- *Fakhroutdinov, K.* (n.d.). *UML class diagrams—Graphical notation reference*. UML diagrams. https://www.uml-diagrams.org/class-reference.html
- *Jain, A.* (2025, August 28). *Unified Modeling Language (UML) diagrams*. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/
- Benjamin, M. (2025, January 19). *The importance of class diagrams in software development*. ResearchGate. https://www.researchgate.net/publication/388162228_The_Importance_of_Class_Diagrams_in_Software_Development
- Bergström, G., Hujainah, F., Ho-Quang, T., Jolak, R., Rukmono, S. A., Nurwidyantoro, A., & Chaudron, M. R. (2022). Evaluating the layout quality of UML class diagrams using machine learning. *Journal of Systems and Software, 192*, 111413. https://doi.org/10.1016/j.jss.2022.111413
- Oloruntoba, S., Walia, A. S., & Kurup, M. (2025, June 12). *SOLID design principles explained: Building better software architecture*. DigitalOcean. https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design
- Lee, P. (2020, June 20). *SOLID principles every programmer should know*. Medium. https://www.medium.com/an-idea/solid-principles-every-programmer-should-know-3399eb663ad2
- Watson, J. (2023, January 21). *Object-oriented programming and design: A practitioner's perspective*. Medium. https://www.medium.com/@james._.watson/object-oriented-programming-and-design-a-practitioners-perspective-3a664cef00b2
- Elsevier. (n.d.). *Object-Oriented Design* [Topic overview]. ScienceDirect Topics. Retrieved November 12, 2025, from https://www.sciencedirect.com/topics/computer-science/object-oriented-design
- Thennakoon, R. D. A., & Hettige, B. (2022, November 23). *A study on object-oriented design principles and patterns*. ResearchGate. https://www.researchgate.net/publication/365683164_A_STUDY_ON_OBJECT-ORIENTED_DESIGN_PRINCIPLES_AND_PATTERNS